

4.BRUTE-FORCE TECHNIQUE:

```
#include <stdio.h>
```

```
#include <math.h>
```

```
struct Point {
```

```
    int x, y;
```

```
};
```

```
float distance(struct Point p1, struct Point p2) {
```

```
    return sqrt((p1.x - p2.x) * (p1.x - p2.x) +
```

```
                (p1.y - p2.y) * (p1.y - p2.y));
```

```
}
```

```
int main() {
```

```
    int n, i, j;
```

```
    struct Point p[100];
```

```
    printf("Enter number of points: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the coordinates (x y):\n");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d %d", &p[i].x, &p[i].y);
```

```
    }
```

```
    float minDist = distance(p[0], p[1]);
```

```
    int p1 = 0, p2 = 1;
```

```
    for (i = 0; i < n; i++) {
```

```

        for (j = i + 1; j < n; j++) {
            float d = distance(p[i], p[j]);
            if (d < minDist) {
                minDist = d;
                p1 = i;
                p2 = j;
            }
        }
    }

    printf("\nClosest Pair: (%d,%d) and (%d,%d)\n",
           p[p1].x, p[p1].y, p[p2].x, p[p2].y);

    printf("Minimum Distance = %.2f\n", minDist);

    return 0;
}

```

OUTPUT:

main.c	Run	Output
<pre> 1 #include <stdio.h> 2 #include <math.h> 3 4- struct Point { 5 int x, y; 6 }; 7 8- float distance(struct Point p1, struct Point p2) { 9 return sqrt((p1.x - p2.x) * (p1.x - p2.x) + 10 (p1.y - p2.y) * (p1.y - p2.y)); 11 } 12 13- int main() { 14 int n, i, j; 15 struct Point p[100]; 16 17 printf("Enter number of points: "); 18 scanf("%d", &n); 19 20 printf("Enter the coordinates (x y):\n"); 21- for (i = 0; i < n; i++) { 22 scanf("%d %d", &p[i].x, &p[i].y); </pre>	Run	<pre> Enter number of points: 4 Enter the coordinates (x y): 2 3 12 30 40 50 1 3 Closest Pair: (2,3) and (1,3) Minimum Distance = 1.00 === Code Execution Successful === </pre>